

PÓS-GRADUAÇÃO EM DESENVOLVIMENTO FULL STACK

SWEET WAY ENGLISH

ISAC NUNES ALVES

Orientador: Júlio Henrique Araújo Pereira Machado

2026

SUMÁRIO

1. CONTEXTUALIZAÇÃO DA PROPOSTA	3
2. OBJETIVOS DA CONSTRUÇÃO DA SOLUÇÃO	4
2.1. Objetivo Estratégico:	4
2.2. Objetivos Específicos:	4
3. ELABORAÇÃO DA JORNADA DO USUÁRIO	5
3.1. O Aluno:	5
3.2. O Professor (e Administrador de Turma):	5
4. APELO MERCADOLÓGICO DA SOLUÇÃO	6
5. CICLO DE DESENVOLVIMENTO DA SOLUÇÃO	8
6. MOCKUP DA PROPOSTA DE SOLUÇÃO	9
6.1. Interfaces do aluno	9
6.2. Interfaces do professor	12
7. ARQUITETURA DE SOFTWARE	15
7.1. Diagramas de contêiner e componentes	15
7.2. Diagrama de implantação	16
7.3. Modelo entidade-relacionamento (Banco de Dados)	17
7.4. Definições dos serviços REST	19
8. VALIDAÇÃO DA SOLUÇÃO	21
9. REGISTROS DAS EVIDÊNCIAS DO PROJETO	22
10. CONSIDERAÇÕES FINAIS E EXPECTATIVAS	23
REFERÊNCIAS	24

1. CONTEXTUALIZAÇÃO DA PROPOSTA

O ensino de idiomas presencial enfrenta frequentemente o desafio de estender a interação da sala de aula para o ambiente digital de forma contínua e engajadora. Recentemente, ao vivenciar o ambiente de uma turma básica de inglês, identificou-se uma lacuna real: a dificuldade de centralizar materiais, áudios e feedbacks num único canal que fosse acessível tanto para alunos quanto para o corpo docente.

No cenário atual de tecnologia educacional, existem diversas soluções com propósitos similares. Encontram-se desde Sistemas de Gestão da Aprendizagem (LMS) robustos e generalistas, como o Moodle e o Google Classroom, até aplicativos focados no autoaprendizado gamificado, como o Duolingo e o Babbel. Contudo, observa-se que ferramentas LMS tradicionais podem ser complexas e burocráticas para professores particulares e pequenas escolas, enquanto os aplicativos de gamificação não se conectam à dinâmica de uma aula presencial específica.

Dessa forma, a proposta deste trabalho é o desenvolvimento da aplicação web Sweet Way English. A solução visa preencher a lacuna entre essas duas abordagens, resolvendo a falta de centralização de materiais e promovendo a interação direta das turmas de idiomas. A ferramenta auxiliará os alunos na prática contínua de exercícios e o professor na gestão do conteúdo e acompanhamento do progresso de forma simples e direta.

2. OBJETIVOS DA CONSTRUÇÃO DA SOLUÇÃO

Os objetivos deste projeto dividem-se em estratégicos e específicos.

2.1. Objetivo Estratégico:

Consolidar a aplicação prática das competências de Desenvolvimento Full Stack, arquitetando e implementando uma solução de ponta a ponta (MVP funcional) que integre interfaces modernas, regras de negócio robustas e persistência de dados eficiente, simulando um cenário real de mercado.

2.2. Objetivos Específicos:

- Desenvolver uma API RESTful utilizando Node.js e TypeScript para gerir turmas, alunos e conteúdos;
- Implementar uma interface responsiva e dinâmica com Next.js, garantindo uma boa experiência de navegação (UX);
- Estruturar um banco de dados relacional (PostgreSQL) utilizando Prisma ORM para garantir a integridade das informações de matrículas e atividades;
- Configurar um ambiente de desenvolvimento e produção padronizado através do uso de Docker.

3. ELABORAÇÃO DA JORNADA DO USUÁRIO

A solução foi pensada para atender dois perfis principais de usuários, com jornadas distintas, mas complementares:

3.1. O Aluno:

A jornada do aluno inicia-se com o login na plataforma, onde tem acesso ao seu "Painel de Turma". Neste ambiente, ele pode visualizar as lições pendentes, acessar materiais de apoio (PDFs, links) e realizar atividades práticas, como o envio de áudios para treino de pronúncia. O fluxo encerra-se com o recebimento do feedback do professor diretamente na atividade.

3.2. O Professor (e Administrador de Turma):

Nesta solução, o professor assume também o papel de gestor do seu ambiente educacional. A jornada inicia-se com o acesso a um Dashboard Administrativo que resume o total de alunos e as tarefas pendentes de correção. É o professor quem realiza a criação de novas turmas na plataforma e gera os códigos de convite (ex: SW-B1-2026), que são compartilhados com os estudantes para que eles próprios se associem à sala correta. Além dessa gestão inicial, o sistema permite que o docente faça o upload de arquivos na seção "Materiais de Apoio" e acesse as submissões dos alunos para atribuir notas e comentários qualitativos.

4. APELO MERCADOLÓGICO DA SOLUÇÃO

A Sweet Way English insere-se no mercado de EdTech (Tecnologia Educacional), focando especificamente em pequenos cursos de idiomas e professores particulares que necessitam de uma plataforma própria e otimizada.

Para compreender o apelo mercadológico da solução, é fundamental observar o cenário atual, que se divide predominantemente em dois polos:

1. LMS Tradicionais (Ex: Google Classroom e Moodle): São plataformas genéricas e robustas. Embora atendam ao quesito de centralização de materiais, a interface e a experiência de uso não são otimizadas especificamente para o ensino de idiomas (como o envio de áudios de pronúncia para speaking rápido).
2. Aplicativos Gamificados (Ex: Duolingo e Babbel): São excelentes para a aquisição de vocabulário, porém funcionam como ferramentas B2C (focadas no autoaprendizado do usuário final) e não permitem que um professor insira o cronograma exato da sua aula presencial para gerenciar uma turma.

Diferente dessas aplicações, o Sweet Way English atua como uma ferramenta B2B e B2B2C, apoiando a sala de aula personalizada. A Tabela 1 ilustra as diferenças de funcionalidades entre as plataformas.

Tabela 1 – Comparativo de soluções de mercado

Funcionalidades	Sweet Way English	Google Classroom	Moodle	Duolingo
Foco em apoio presencial	Sim	Sim	Sim	Não
UX simples para pequenas turmas	Alta	Média	Baixa	Alta
Módulos focados em Idiomas (Áudios/Feedbacks)	Sim	Parcial	Parcial	Sim
Gestão e acompanhamento por um professor	Sim	Sim	Sim	Não
Possibilidade de personalização	Sim	Não	Sim	Não

Fonte: Elaborado pelo autor (2026).

O modelo de negócio sustentável baseia-se na oferta da plataforma no formato SaaS (Software as a Service), cobrando uma mensalidade acessível para que professores e escolas de bairro possam digitalizar o ensino sem perder a identidade e a proximidade da sua turma presencial.

5. CICLO DE DESENVOLVIMENTO DA SOLUÇÃO

O desenvolvimento seguiu uma abordagem iterativa e incremental, típica de metodologias ágeis. O ciclo foi dividido nas seguintes etapas:

1. Conceção e Design: Definição dos requisitos e prototipagem das telas;
2. Configuração de Ambiente: Setup do Docker e repositórios;
3. Desenvolvimento Backend: Implementação da API, Models do Prisma e Migrations;
4. Desenvolvimento Frontend: Integração das telas com a API;
5. Testes e Refinamento: Validação dos fluxos de cadastro e submissão de atividades.

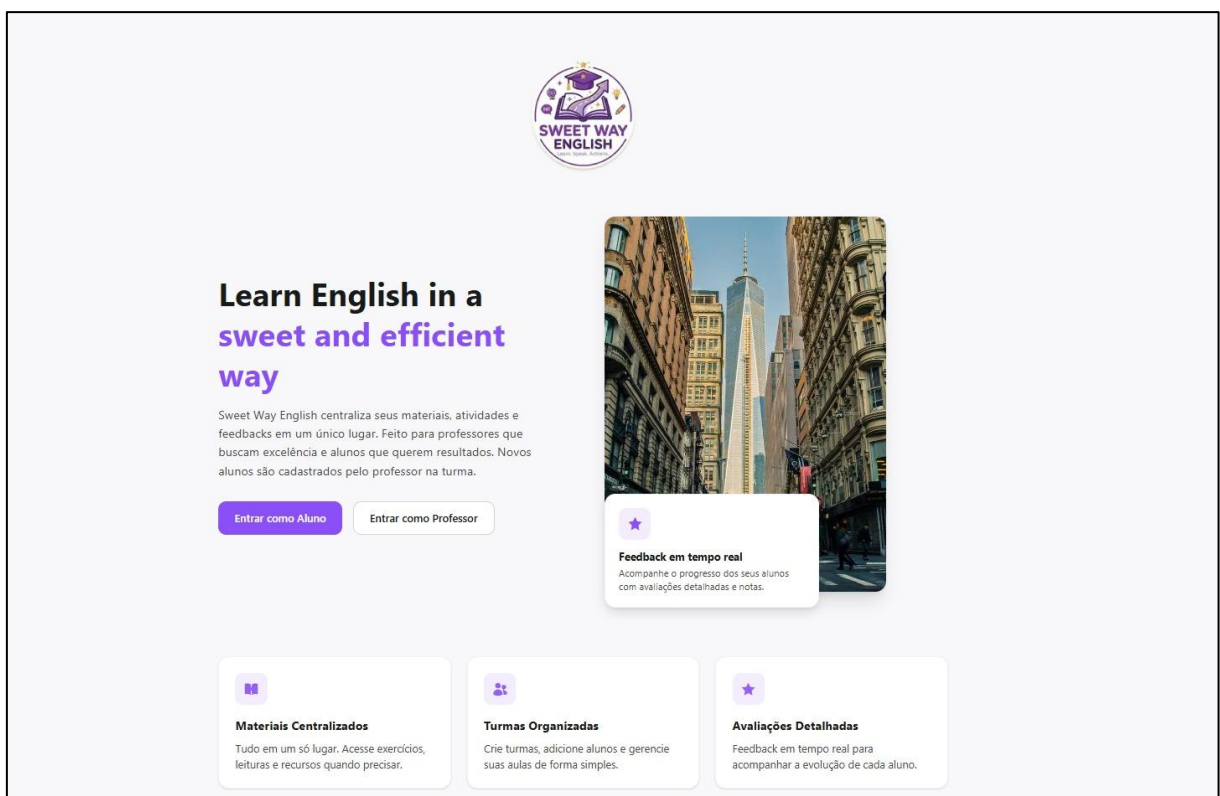
Para garantir que não houvesse conflito de versões, utilizou-se o Git com separação clara de responsabilidades no código.

6. MOCKUP DA PROPOSTA DE SOLUÇÃO

Abaixo são apresentadas as interfaces finais desenvolvidas para a solução, demonstrando a aplicação prática dos conceitos de UX/UI. As telas estão organizadas em dois grupos: interfaces voltadas ao perfil de aluno e interfaces voltadas ao perfil de professor.

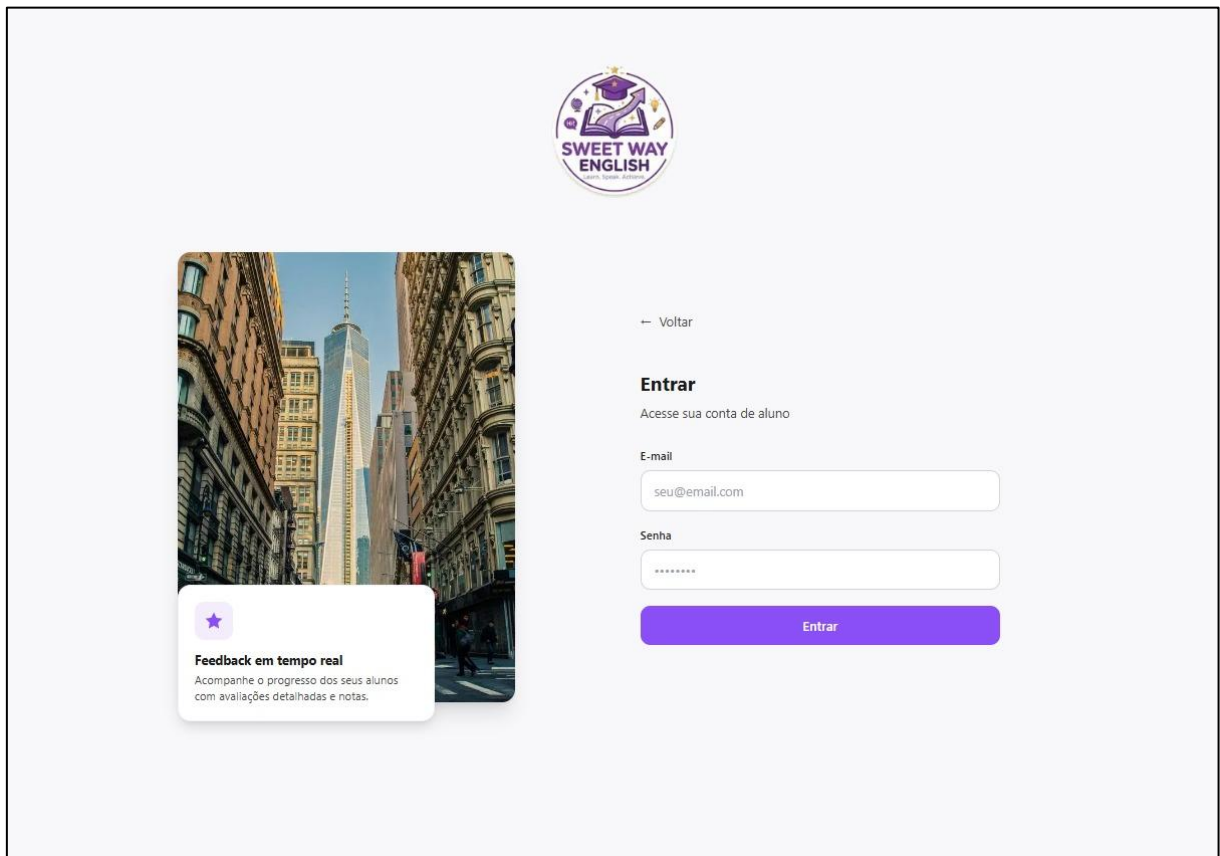
6.1. Interfaces do aluno

Figura 1 - Página inicial da plataforma



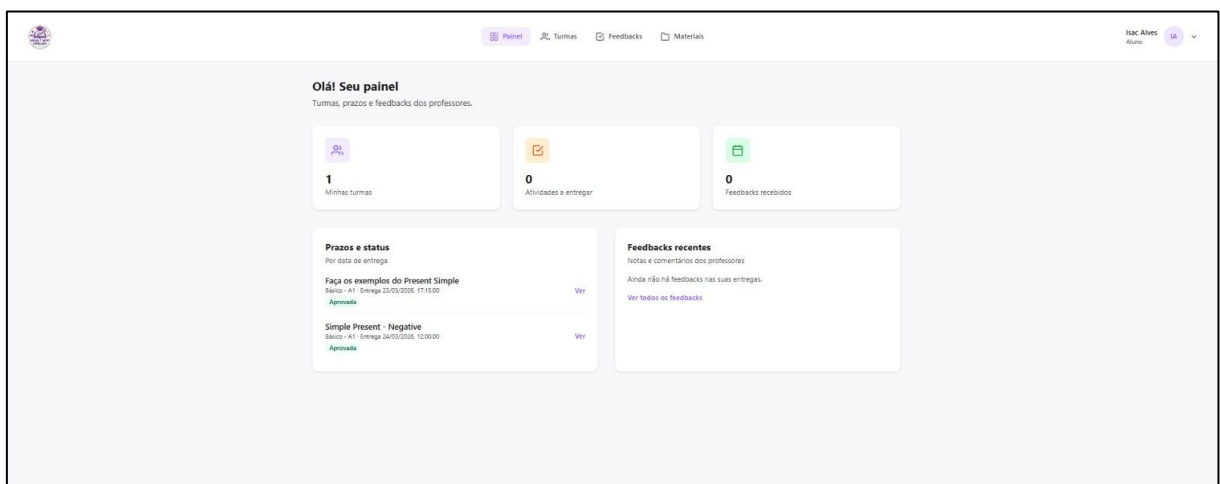
Fonte: Elaborado pelo autor (2026).

Figura 2 – Tela de login do aluno



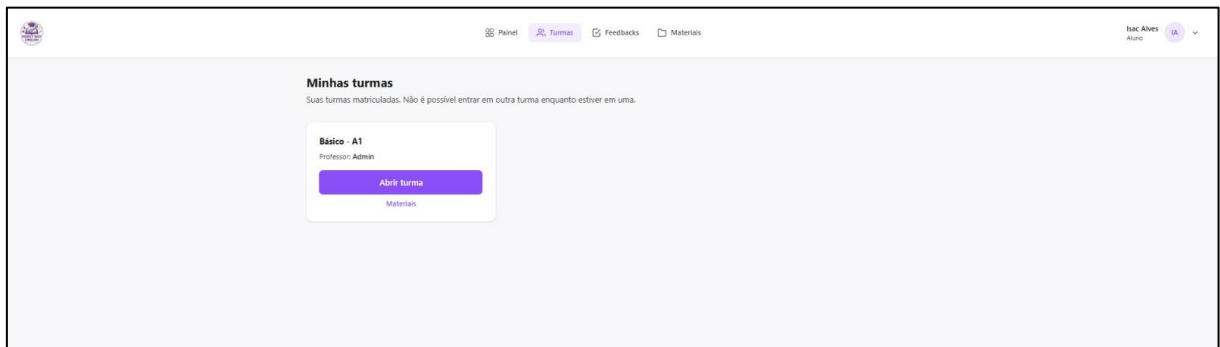
Fonte: Elaborado pelo autor (2026).

Figura 3 – Painel do aluno



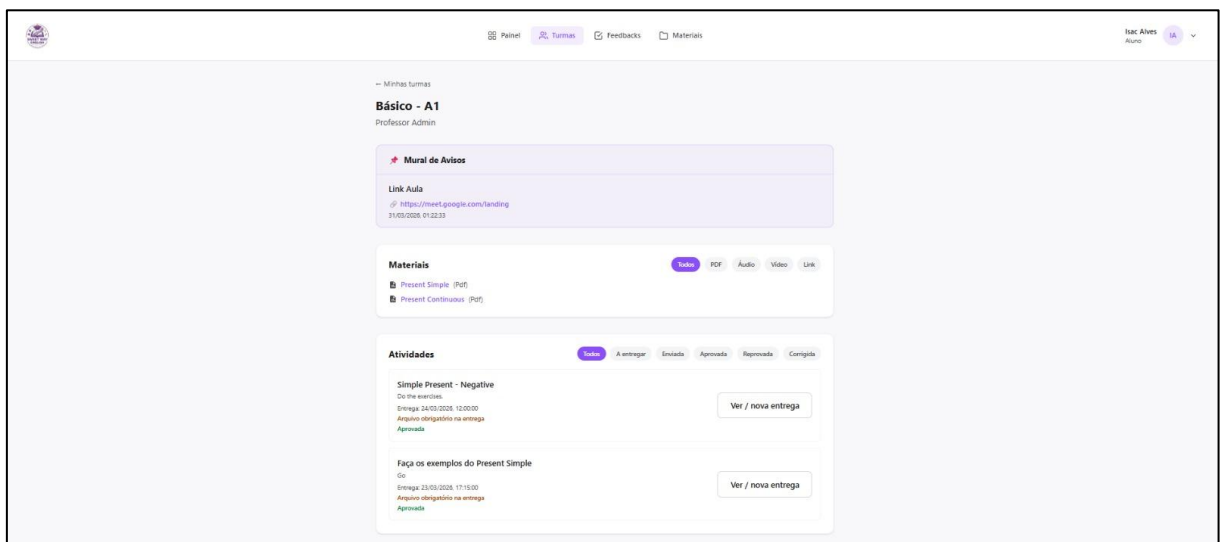
Fonte: Elaborado pelo autor (2026).

Figura 4 – Listagem de turmas do aluno



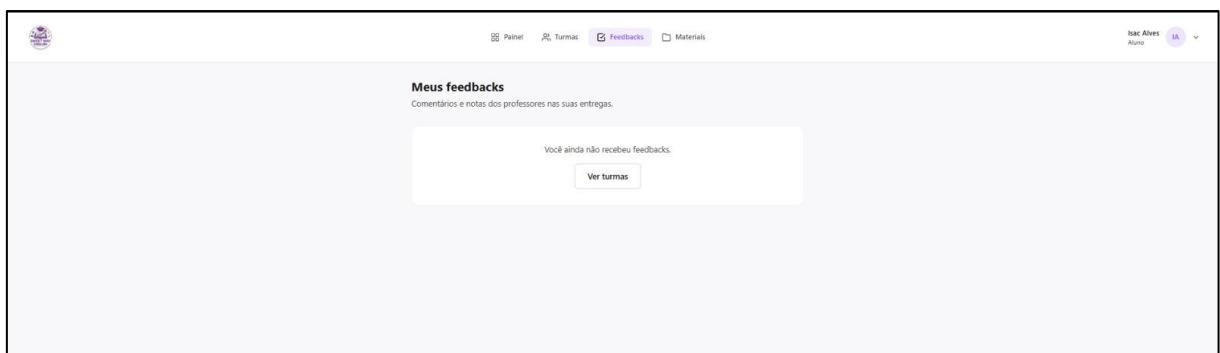
Fonte: Elaborado pelo autor (2026).

Figura 5 – Detalhe da turma



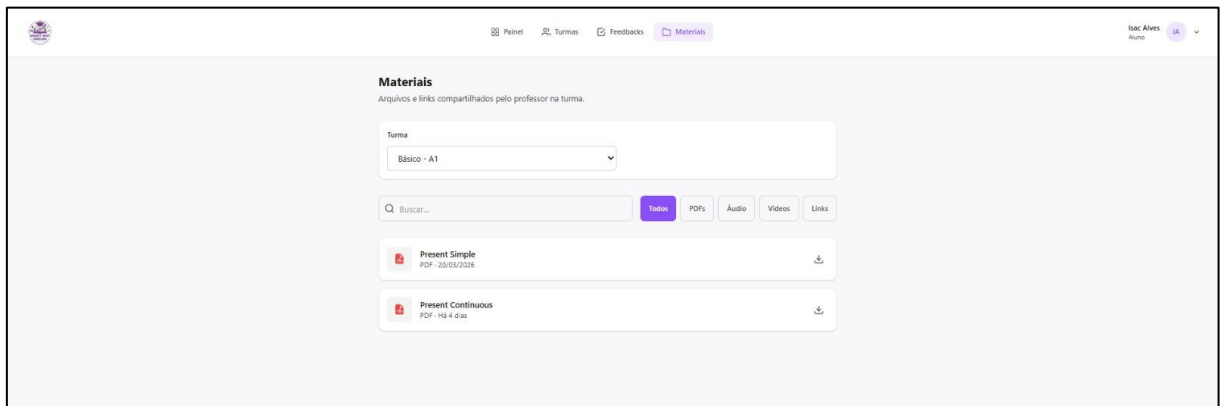
Fonte: Elaborado pelo autor (2026).

Figura 6 – Feedbacks do aluno



Fonte: Elaborado pelo autor (2026).

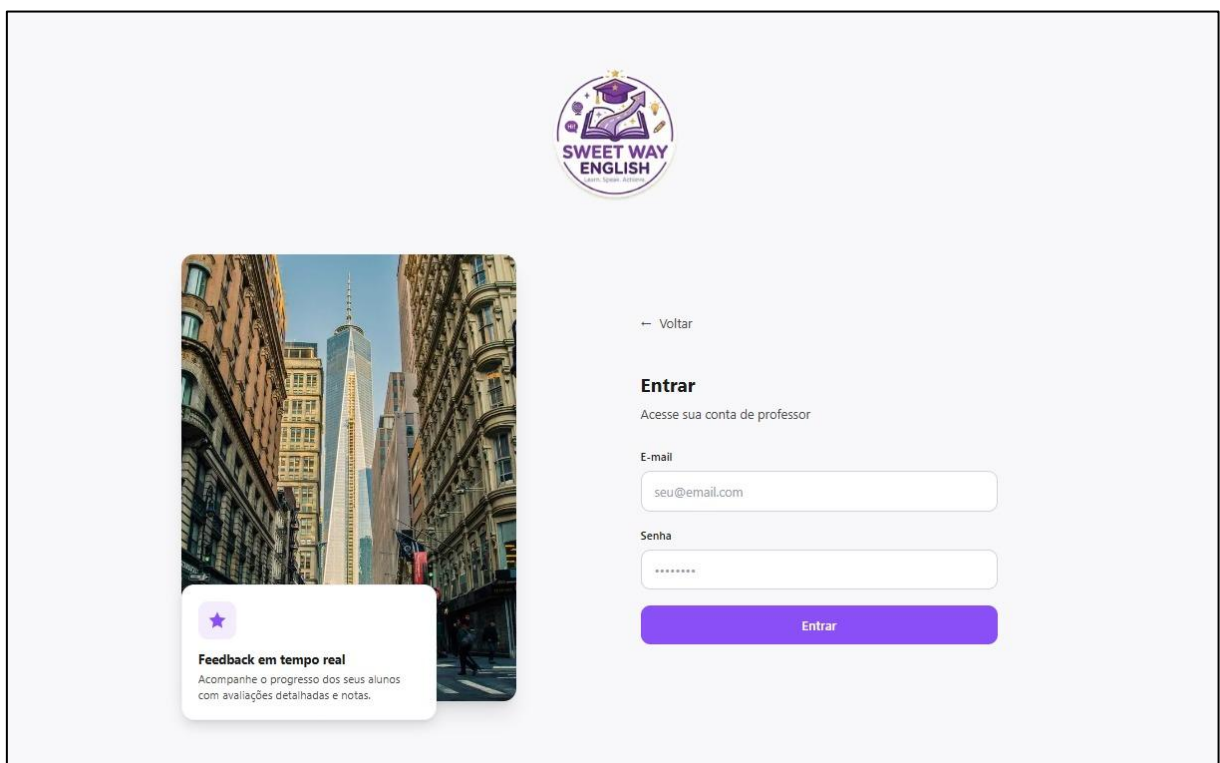
Figura 7 – Materiais de apoio



Fonte: Elaborado pelo autor (2026).

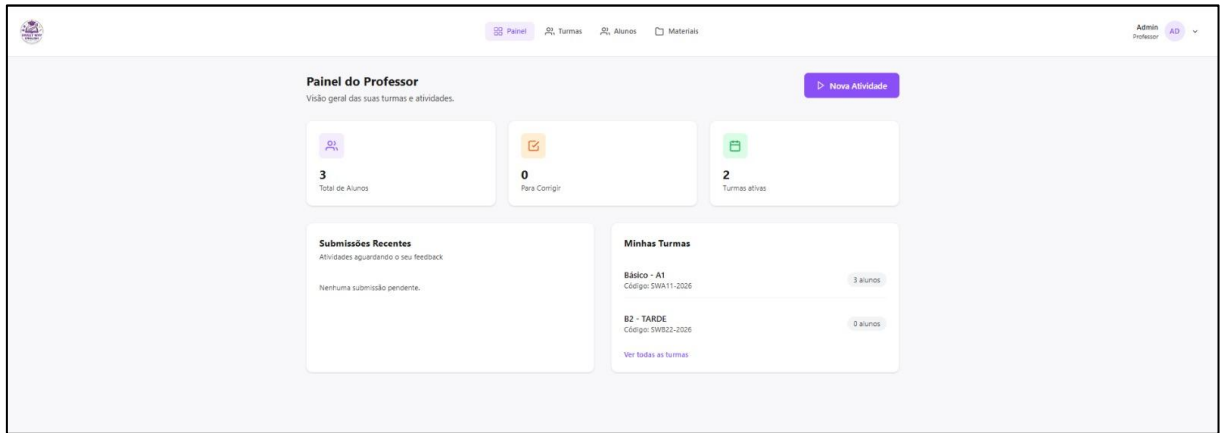
6.2. Interfaces do professor

Figura 8 – Tela de login do professor



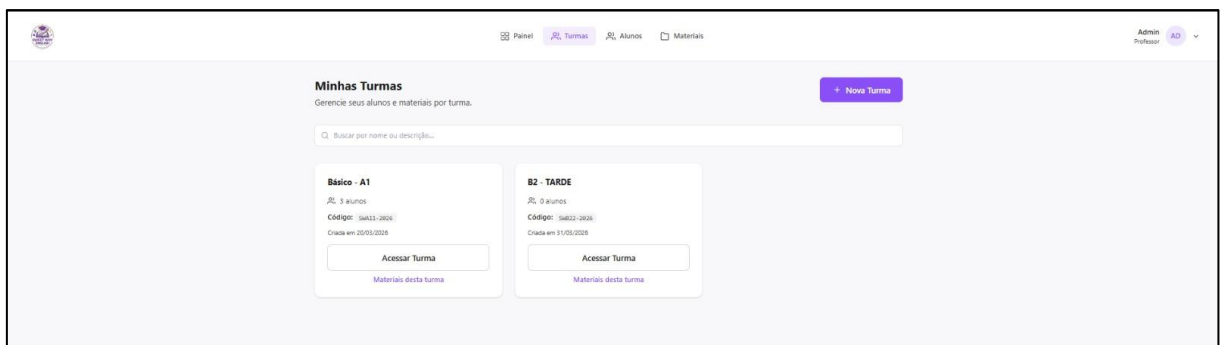
Fonte: Elaborado pelo autor (2026).

Figura 9 – Painel do professor



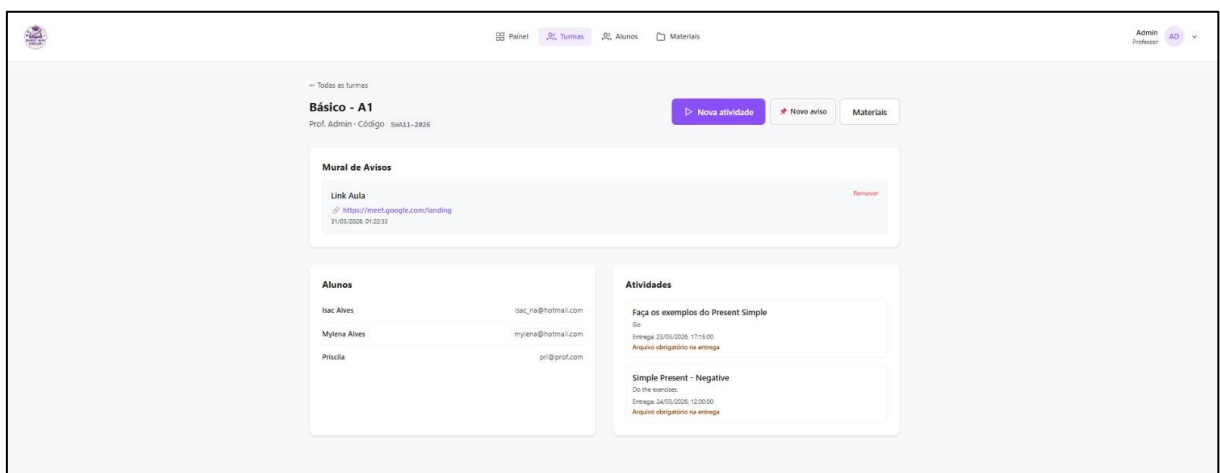
Fonte: Elaborado pelo autor (2026).

Figura 10 – Gerenciamento de turmas



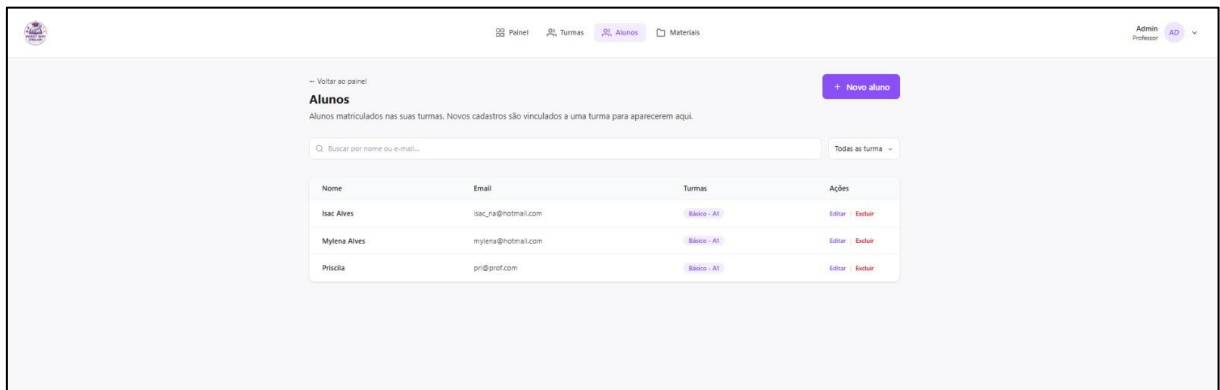
Fonte: Elaborado pelo autor (2026).

Figura 11 – Detalhe da turma



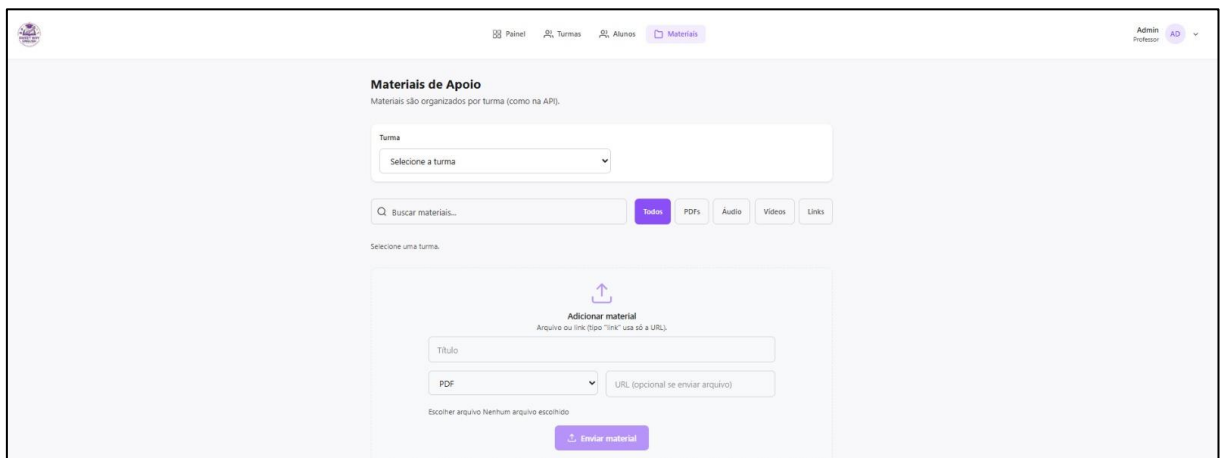
Fonte: Elaborado pelo autor (2026).

Figura 12 – Gestão de alunos



Fonte: Elaborado pelo autor (2026).

Figura 13 – Materiais de apoio



Fonte: Elaborado pelo autor (2026).

7. ARQUITETURA DE SOFTWARE

A arquitetura da solução foi projetada para garantir desacoplamento, escalabilidade e facilidade de manutenção (MARTIN, 2019). Para a representação estrutural do sistema, adotou-se a padronização de diagramas baseada no Modelo C4, que permite visualizar a solução em diferentes níveis de abstração.

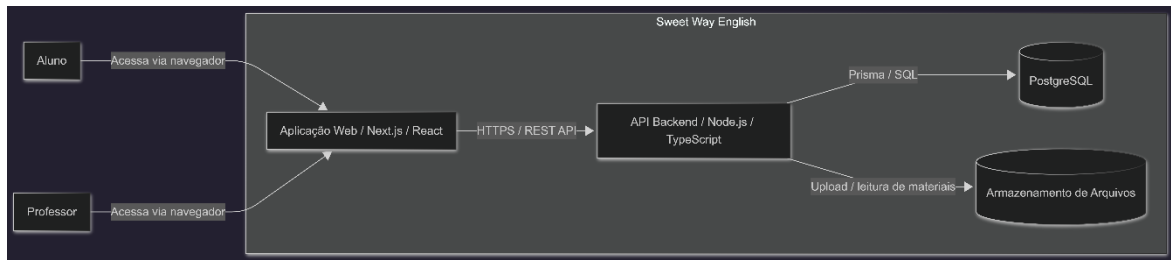
- Camada de Interface (Frontend): Construída em Next.js (VERCEL, 2026) e React (FACEBOOK, 2026), sendo responsável pela renderização das páginas e gestão de rotas no lado do cliente.
- Camada de Aplicação (Backend): Desenvolvida em Node.js (NODE.JS FOUNDATION, 2026) com TypeScript (MICROSOFT, 2026), servindo como API REST para o frontend. Garante a tipagem estática e segurança no tráfego de dados.
- Camada de Persistência (Banco de Dados): Utilização do PostgreSQL gerido pelo Prisma ORM (PRISMA, 2026).
- Infraestrutura (DevOps): Todo o sistema é encapsulado em contêineres Docker (DOCKER, 2026).

7.1. Diagramas de contêiner e componentes

O diagrama apresentado na Figura 14 foi elaborado com base no modelo C4, contemplando os níveis de contêiner e componentes. No nível de contêiner, são representados os principais blocos da aplicação Sweet Way English, incluindo a aplicação web em Next.js, a API backend em Node.js com TypeScript e o banco de dados PostgreSQL, além de um possível contêiner de armazenamento de arquivos. A comunicação entre esses elementos ocorre via HTTP/HTTPS e consultas SQL, garantindo a separação de responsabilidades e o desacoplamento da solução.

No nível de componentes, detalha-se a organização interna do backend, estruturado em controladores, serviços e repositórios, apoiando-se em padrões de projeto consolidados (FREEMAN; ROBSON, 2021), e seguindo princípios de arquitetura limpa (MARTIN, 2019). Essa abordagem favorece a manutenção, testabilidade e escalabilidade do sistema, conforme detalhado na Figura 14.

Figura 14 – Diagrama de contêineres da aplicação

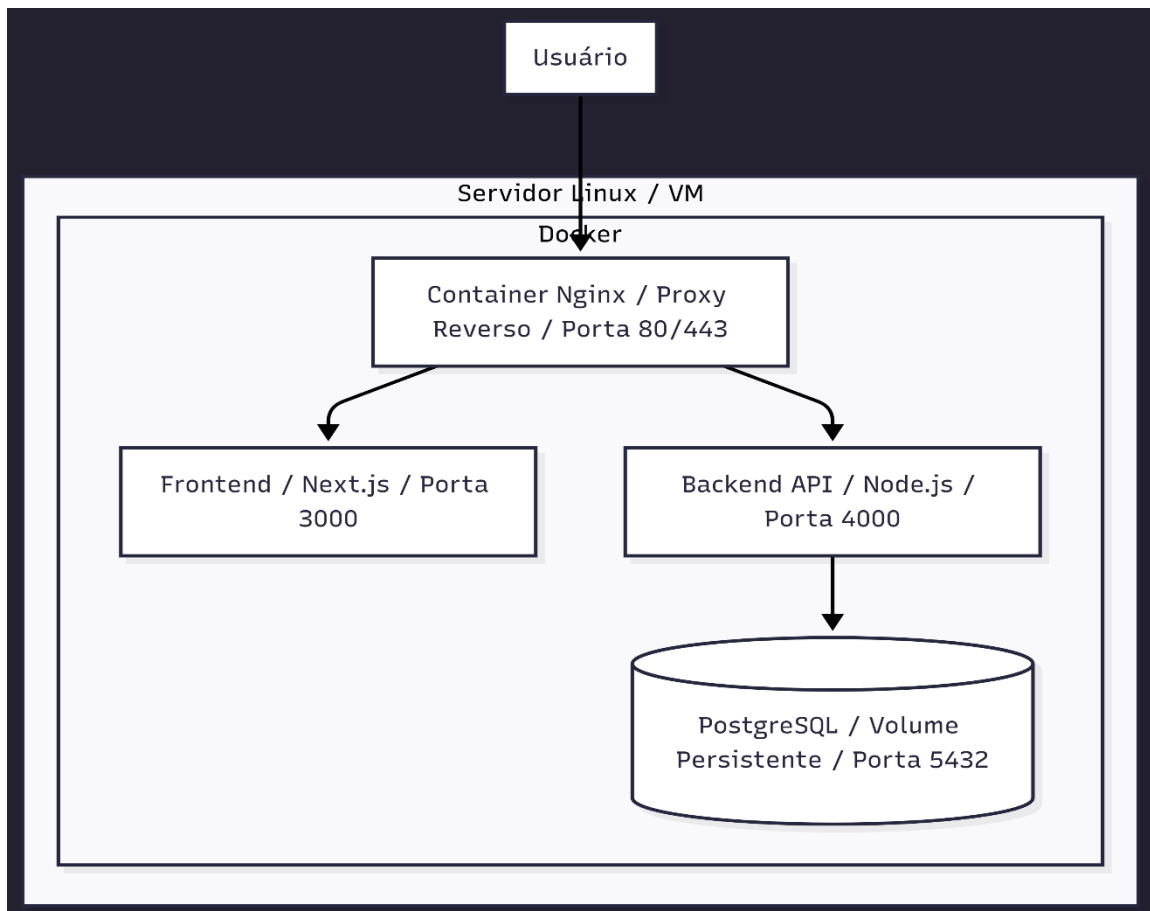


Fonte: Elaborado pelo autor (2026).

7.2. Diagrama de implantação

A infraestrutura da aplicação foi projetada com base no paradigma de containerização, utilizando Docker para garantir consistência entre os ambientes de desenvolvimento e produção. O diagrama de implantação apresentado na Figura 15 apresenta a distribuição dos serviços em contêineres isolados, evidenciando a comunicação entre frontend, backend e banco de dados, bem como as portas e protocolos utilizados para interação entre os componentes.

Figura 15 – Diagrama de implantação



Fonte: Elaborado pelo autor (2026).

7.3. Modelo entidade-relacionamento (Banco de Dados)

Para suportar as regras de negócio da plataforma Sweet Way English, a base de dados foi modelada de forma relacional, conforme representado na Figura 16, com foco na integridade referencial, na rastreabilidade das interações acadêmicas e na separação clara das responsabilidades de cada entidade. O modelo contempla os principais atores e objetos do domínio educacional, permitindo representar professores, alunos, turmas, atividades, materiais de apoio, matrículas e submissões realizadas pelos estudantes.

A entidade **Professor** representa o responsável pedagógico pela criação e gestão das turmas. Cada professor pode estar associado a uma ou mais turmas. A entidade **Turma** funciona como núcleo organizacional da aplicação, concentrando os alunos matriculados, as atividades propostas e os materiais disponibilizados.

A relação entre **Aluno** e **Turma** é do tipo muitos-para-muitos, sendo resolvida pela entidade associativa **Matrícula**, que registra o vínculo formal do estudante com a turma. Essa abordagem permite controlar a participação dos alunos em diferentes turmas e manter histórico acadêmico de forma consistente.

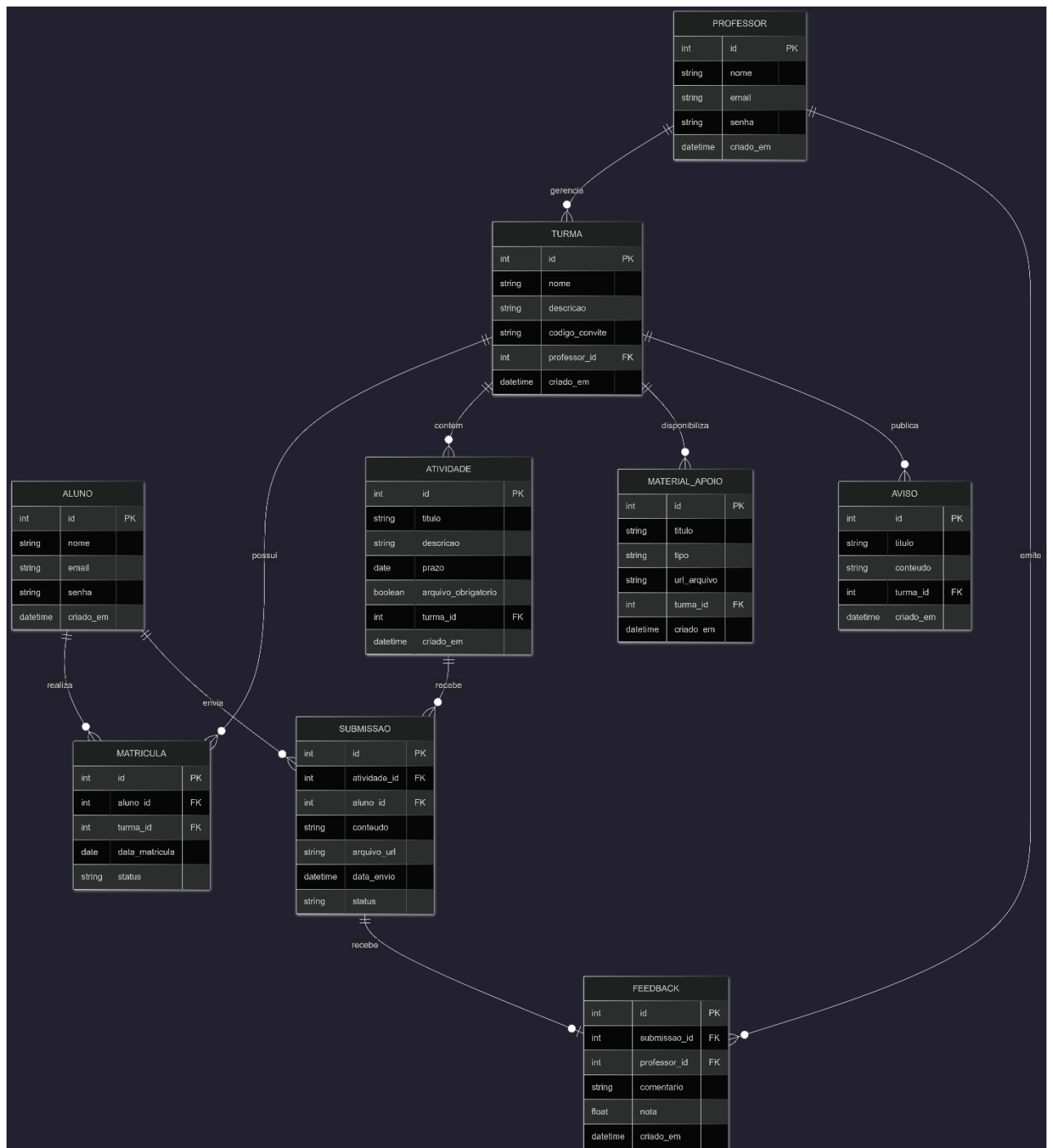
A entidade **Atividade** armazena os exercícios propostos pelo professor para uma turma específica, contendo informações como prazo de entrega (prazo) e indicação de obrigatoriedade de arquivo anexo. Cada atividade pode receber uma ou mais Submissões, realizadas pelos alunos matriculados. As submissões representam as entregas dos estudantes, como respostas textuais ou arquivos de áudio, e possuem um ciclo de status definido: pendente, aprovado, reprovado ou corrigido.

Para registrar o retorno pedagógico, cada submissão pode estar vinculada a uma entidade de **Feedback**, responsável por armazenar comentários, observações e notas atribuídas pelo professor. Além disso, a entidade **Material de Apoio** permite associar arquivos e conteúdos complementares às turmas, como PDF, links, áudios e vídeos utilizados durante o processo de ensino-aprendizagem.

A entidade **Aviso** permite que o professor publique comunicados e informações relevantes diretamente na turma, mantendo os alunos informados sobre atualizações do curso sem depender de canais externos de comunicação.

Dessa forma, o modelo entidade-relacionamento proposto assegura consistência estrutural, reduz redundâncias e oferece suporte adequado às principais operações da plataforma, como gestão de turmas, acompanhamento discente, publicação de atividades e avaliação de desempenho.

Figura 16 – Diagrama entidade-relacionamento



Fonte: Elaborado pelo autor (2026).

7.4. Definições dos serviços REST

A API da plataforma Sweet Way English foi desenvolvida seguindo os princípios da arquitetura REST (Representational State Transfer), utilizando os verbos HTTP de forma semântica para representar as operações sobre os recursos do sistema. As rotas estão organizadas em arquivos de controller independentes — autenticação, turmas, atividades, avisos, materiais, submissões, feedbacks, alunos e administração —, promovendo a separação de responsabilidades e facilitando a manutenção da aplicação (MARTIN, 2019).

A segurança dos endpoints é garantida pelo middleware de autenticação JWT (authenticateToken), que valida o token Bearer em cada requisição protegida e anexa o perfil do usuário (id e role) ao contexto da requisição. Rotas exclusivas de professor ou de aluno são protegidas adicionalmente por verificação de papel (role), impedindo acessos cruzados entre perfis.

A Tabela 2 apresenta os principais endpoints da API, organizados por controller, com seus respectivos verbos de acesso e funcionalidades de negócio associadas.

Tabela 2 – Endpoints da API REST

Controller	Verbo	URL	Funcionalidade de negócio
Autenticação	POST	/api/auth/register/professor	Cadastro de novo professor
	POST	/api/auth/register/aluno	Cadastro de novo aluno
	POST	/api/auth/login	Autenticação e geração de token JWT
Turmas	POST	/api/turmas	Criação de turma com código de convite
	GET	/api/turmas	Listagem das turmas do usuário autenticado
	GET	/api/turmas/:id	Detalhes da turma com atividades, materiais e avisos
	POST	/api/turmas/entrar	Matrícula de aluno via código de convite
Atividades	POST	/api/turmas/:turmaId/atividades	Publicação de atividade em uma turma
	GET	/api/turmas/:turmaId/atividades	Listagem de atividades de uma turma
	PATCH	/api/professor/atividades/:id	Atualização de atividade
Avisos	GET	/api/turmas/:turmaId/avisos	Listagem de avisos de uma turma
	POST	/api/turmas/:turmaId/avisos	Criação de aviso
	DELETE	/api/turmas/:turmaId/avisos/:avisoid	Remoção de aviso
Materiais	GET	/api/turmas/:id/materiais	Listagem dos materiais de apoio de uma turma
	POST	/api/turmas/:id/materiais	Upload de material de apoio

Controller	Verbo	URL	Funcionalidade de negócio
Submissões	DELETE	/api/turmas/:id/materiais/:materialId	Remoção de material de apoio
	POST	/api/atividades/:id/submissoes	Envio de submissão pelo aluno (upload de arquivo)
	GET	/api/atividades/:id/submissoes	Listagem de submissões de uma atividade
Feedbacks	PATCH	/api/professor/submissoes/:id	Atualização do status de correção
	POST	/api/submissoes/:id/feedback	Registro de feedback com nota e comentário
	GET	/api/alunos/me/feedbacks	Consulta dos feedbacks recebidos pelo aluno autenticado
Alunos	GET	/api/professor/alunos	Listagem de alunos
	POST	/api/professor/alunos	Cadastro de aluno
	PATCH	/api/professor/alunos/:id	Atualização de dados de aluno
Administração	DELETE	/api/professor/alunos/:id	Remoção de aluno
	GET	/api/configuracoes	Consulta das configurações da aplicação
	PATCH	/api/configuracoes/logo	Upload do logotipo da plataforma
	GET	/api/professor/me	Consulta do perfil do professor autenticado
	PATCH	/api/professor/me	Atualização do perfil do professor autenticado
	GET	/api/admin/professores	Listagem de todos os professores
	POST	/api/admin/professores	Cadastro de professor
	PATCH	/api/admin/professores/:id	Atualização de professor
	DELETE	/api/admin/professores/:id	Remoção de professor
	POST	/api/admin/alunos/:id/promover	Promoção de aluno para professor
Health	GET	/api/health	Verificação de integridade da API

Fonte: Elaborado pelo autor (2026).

8. VALIDAÇÃO DA SOLUÇÃO

A garantia da qualidade do software foi realizada de forma pragmática através da execução de testes funcionais manuais ao longo de todo o ciclo de desenvolvimento. Optou-se pela validação manual para garantir a agilidade na entrega do Produto Viável Mínimo (MVP), testando exaustivamente cada rota da API via ferramentas cliente (como o Postman) e cada interação do utilizador na interface gráfica.

Além da validação manual, a integridade do código é assegurada preventivamente pelo uso do TypeScript (MICROSOFT, 2026), que acusa erros de tipagem em tempo de compilação, e pelas constraints (restrições) do banco de dados relacional. A validação final consistiu em um teste integrado simulando o uso real (end-to-end manual): criação de turma pelo professor, cadastro de um aluno, matrícula e o fluxo completo de entrega e correção de uma atividade.

9. REGISTROS DAS EVIDÊNCIAS DO PROJETO

Todo o código-fonte desenvolvido, bem como a documentação técnica de instalação e execução, encontra-se disponível publicamente no repositório de versionamento.

- Repositório GitHub: <https://github.com/isacna/sweet-way-english.git>
- Documentação: Disponível no ficheiro README.md do repositório.

10. CONSIDERAÇÕES FINAIS E EXPECTATIVAS

A elaboração do Sweet Way English permitiu consolidar a compreensão de que o desenvolvimento Full Stack vai além da implementação de funcionalidades, envolvendo a construção de soluções coesas, escaláveis e orientadas à experiência do usuário.

Durante o desenvolvimento da solução, algumas dificuldades técnicas relevantes foram enfrentadas. O principal desafio inicial consistiu na configuração de um ambiente de desenvolvimento unificado e na comunicação em rede entre os contêineres do frontend, backend e banco de dados. Essa dificuldade foi superada por meio da adoção de práticas de containerização e redes internas utilizando Docker, garantindo isolamento e padronização do ambiente (DOCKER, 2026).

Outro desafio significativo foi a modelagem adequada da base de dados, especialmente na representação do relacionamento entre alunos e turmas. A necessidade de permitir que um aluno participasse de múltiplas turmas sem redundância exigiu a utilização de relacionamentos muitos-para-muitos, implementados com o suporte do Prisma ORM (PRISMA, 2026) sobre o banco de dados PostgreSQL (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026).

Do ponto de vista arquitetural, a separação entre frontend, backend e camada de persistência, aliada ao uso de uma API estruturada em Node.js com TypeScript (NODE.JS FOUNDATION, 2026; MICROSOFT, 2026), permitiu a construção de uma solução desacoplada e de fácil manutenção, alinhada a princípios de arquitetura limpa (MARTIN, 2019).

A principal evolução profissional observada foi a transição de uma abordagem estritamente técnica para uma visão orientada a produto, considerando aspectos como experiência do usuário, organização arquitetural e escalabilidade da solução. Como perspectiva futura, pretende-se evoluir o MVP para um produto comercializável, validando-o com turmas reais e implementando melhorias contínuas com base no feedback dos utilizadores.

REFERÊNCIAS

DOCKER. Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em: 10 fev. 2026.

FACEBOOK. React: uma biblioteca JavaScript para criar interfaces de usuário. Disponível em: <https://pt-br.reactjs.org/>. Acesso em: 26 jan. 2026.

FREEMAN, Eric; ROBSON, Elisabeth. Head First Design Patterns. 2. ed. Sebastopol: O'Reilly Media, 2021.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL: The World's Most Advanced Open Source Relational Database. Disponível em: <https://www.postgresql.org/>. Acesso em: 10 fev. 2026.

PRISMA. Prisma ORM Documentation. Disponível em: <https://www.prisma.io/docs>. Acesso em: 26 jan. 2026.

MARTIN, Robert C. Arquitetura Limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019

MICROSOFT. TypeScript: Typed JavaScript at Any Scale. Disponível em: <https://www.typescriptlang.org/>. Acesso em: 10 fev. 2026.

NODE.JS FOUNDATION. Node.js Documentation. Disponível em: <https://nodejs.org/en/docs>. Acesso em: 26 jan. 2026.

VERCEL. Next.js Documentation. Disponível em: <https://nextjs.org/docs>. Acesso em: 10 fev. 2026

FIELDING, Roy T. Architectural Styles and the Design of Network-based Software Architectures. Dissertação (Doutorado) — University of California, Irvine, 2000.